

SLIIT
SE3082 – Parallel Computing



REPORT
LAB 05

Akash D
IT24101853

Exercise 04

Code Implementation

 /a/3 Semester/PC/Lab05/Exercise04

GNU nano 9.1

```
#include <omp.h>
```

```
#include <stdio.h>
```

```
int main() {
```

```
#pragma omp parallel for
```

```
  for (int r=1; r<=5; r++)
```

```
    for (int k=1; k<=10; k++) {
```

```
      printf("r = %d, k = %d, thread # = %d\n", r, k, omp_get_thread_num());
```

```
    }
```

```
  return 0;
```

```
}|
```

Output

```
M /a/3 Semester/PC/Lab05/Exercise04  
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise04  
$ gcc -fopenmp lab04d.c -o lab04d.o  
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise04  
$ ./lab04d.o  
r = 2, k = 1, thread # = 1  
r = 2, k = 2, thread # = 1  
r = 2, k = 3, thread # = 1  
r = 2, k = 4, thread # = 1  
r = 2, k = 5, thread # = 1  
r = 2, k = 6, thread # = 1  
r = 2, k = 7, thread # = 1  
r = 2, k = 8, thread # = 1  
r = 2, k = 9, thread # = 1  
r = 2, k = 10, thread # = 1  
r = 5, k = 1, thread # = 4  
r = 5, k = 2, thread # = 4  
r = 5, k = 3, thread # = 4  
r = 5, k = 4, thread # = 4  
r = 5, k = 5, thread # = 4  
r = 5, k = 6, thread # = 4  
r = 5, k = 7, thread # = 4  
r = 5, k = 8, thread # = 4  
r = 5, k = 9, thread # = 4  
r = 5, k = 10, thread # = 4  
r = 4, k = 1, thread # = 3  
r = 4, k = 2, thread # = 3  
r = 4, k = 3, thread # = 3  
r = 4, k = 4, thread # = 3  
r = 4, k = 5, thread # = 3  
r = 4, k = 6, thread # = 3  
r = 4, k = 7, thread # = 3  
r = 4, k = 8, thread # = 3  
r = 4, k = 9, thread # = 3  
r = 4, k = 10, thread # = 3  
r = 3, k = 1, thread # = 2  
r = 3, k = 2, thread # = 2  
r = 3, k = 3, thread # = 2  
r = 1, k = 1, thread # = 0  
r = 1, k = 2, thread # = 0  
r = 1, k = 3, thread # = 0  
r = 1, k = 4, thread # = 0  
r = 1, k = 5, thread # = 0  
r = 1, k = 6, thread # = 0  
r = 1, k = 7, thread # = 0  
r = 1, k = 8, thread # = 0  
r = 1, k = 9, thread # = 0  
r = 1, k = 10, thread # = 0  
r = 3, k = 4, thread # = 2  
r = 3, k = 5, thread # = 2  
r = 3, k = 6, thread # = 2  
r = 3, k = 7, thread # = 2  
r = 3, k = 8, thread # = 2  
r = 3, k = 9, thread # = 2  
r = 3, k = 10, thread # = 2  
LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise04  
$ |
```

How Threads are Generated (Standard `#pragma omp parallel for`) Outer Loop Parallelization Only: By default, OpenMP only parallelizes the outer loop (r from 1 to 5). Inner Loop Sequential: The inner loop (k from 1 to 10) is executed sequentially by whichever thread owns that specific iteration of r. Work Distribution: The 5 outer iterations are distributed across the available threads in the team. Because there are only 5 iterations in the outer loop, if you use more than 5 threads (e.g., 8 threads), some threads will remain idle since there aren't enough outer iterations to go around.

Modify for collapse(2)

 /a/3 Semester/PC/Lab05/Exercise04

GNU nano 9.1

```
#include <omp.h>
#include <stdio.h>

int main() {

#pragma omp parallel for collapse(2)
  for (int r=1; r<=5; r++)
    for (int k=1; k<=10; k++) {
      printf("r = %d, k = %d, thread # = %d\n", r, k, omp_get_thread_num());
    }

  return 0;
}
```

Outputt

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise04

```
$ ./lab04d.o
r = 1, k = 5, thread # = 1
r = 1, k = 6, thread # = 1
r = 1, k = 7, thread # = 1
r = 1, k = 8, thread # = 1
r = 2, k = 8, thread # = 5
r = 2, k = 9, thread # = 5
r = 2, k = 10, thread # = 5
r = 3, k = 1, thread # = 6
r = 3, k = 2, thread # = 6
r = 3, k = 3, thread # = 6
r = 5, k = 5, thread # = 14
r = 5, k = 6, thread # = 14
r = 5, k = 7, thread # = 14
r = 3, k = 10, thread # = 9
r = 4, k = 1, thread # = 9
r = 4, k = 2, thread # = 9
r = 1, k = 9, thread # = 2
r = 1, k = 10, thread # = 2
r = 2, k = 1, thread # = 2
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 3, k = 7, thread # = 8
r = 3, k = 8, thread # = 8
r = 3, k = 9, thread # = 8
r = 4, k = 6, thread # = 11
r = 4, k = 7, thread # = 11
r = 4, k = 8, thread # = 11
r = 4, k = 9, thread # = 12
r = 4, k = 10, thread # = 12
r = 5, k = 1, thread # = 12
r = 5, k = 8, thread # = 15
r = 5, k = 9, thread # = 15
r = 5, k = 10, thread # = 15
r = 2, k = 5, thread # = 4
r = 2, k = 6, thread # = 4
r = 2, k = 7, thread # = 4
r = 3, k = 4, thread # = 7
r = 3, k = 5, thread # = 7
r = 3, k = 6, thread # = 7
r = 4, k = 3, thread # = 10
r = 4, k = 4, thread # = 10
r = 4, k = 5, thread # = 10
r = 2, k = 2, thread # = 3
r = 2, k = 3, thread # = 3
r = 2, k = 4, thread # = 3
r = 5, k = 2, thread # = 13
r = 5, k = 3, thread # = 13
r = 5, k = 4, thread # = 13
```

How Threads are Generated with `#pragma omp parallel for collapse(2)` Combined Iteration Space: The `collapse(2)` clause tells the compiler to combine the nested loops into a single, flattened iteration space. Total Workload Size: Instead of only looking at the 5 outer iterations, the compiler calculates the total combined iterations ($5 \times 10 = 50$ total iterations). Better Load Distribution: All 50 iterations are flattened and distributed evenly among the available threads. This allows OpenMP to utilize more threads effectively (up to 50 threads in this case) and prevents load imbalance, which is especially useful when the outer loop is small.